

Documentation Technique — Frontend

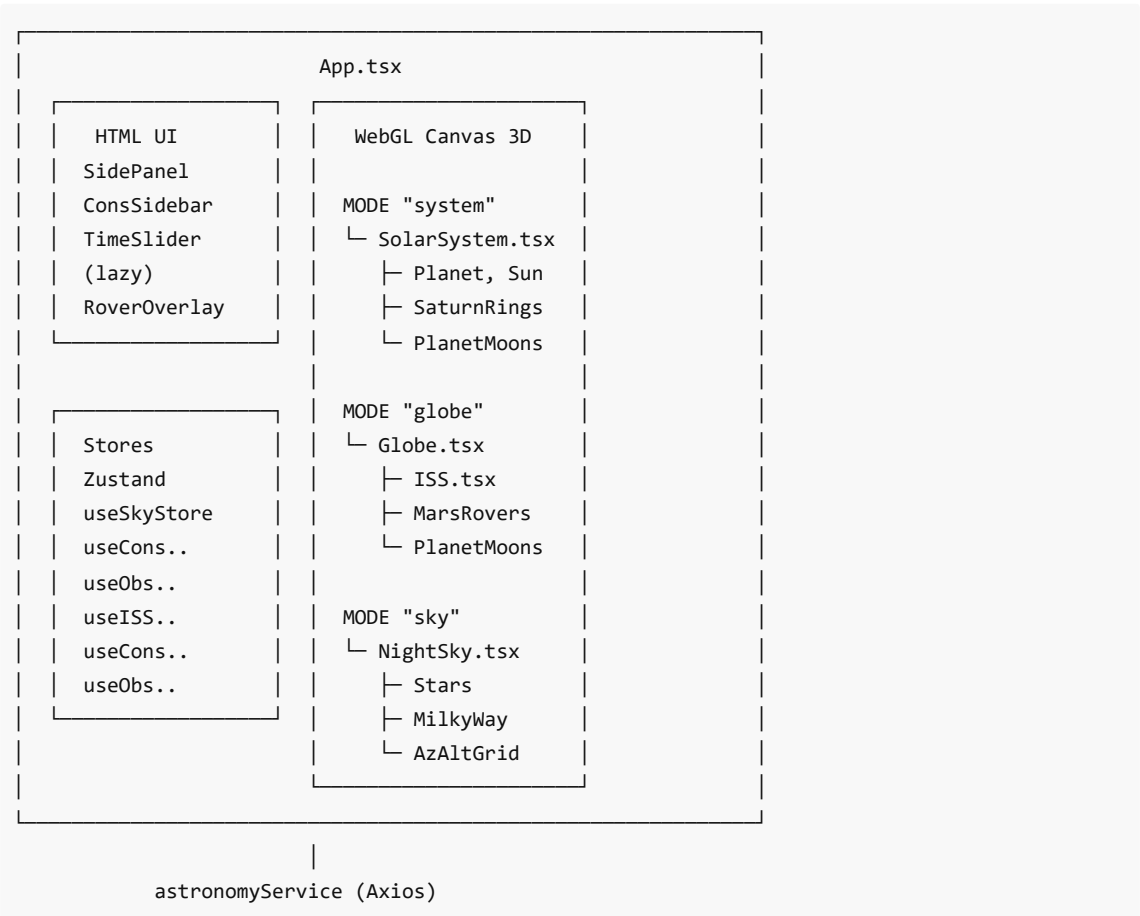
Projet : Night Sky Viewer
Stack : React 19 · TypeScript 5.9 · Vite 7 · Three.js 0.182 · React Three Fiber 9 · Zustand 5
Port dev : http://localhost:5173

Table des matières

- 1. [Architecture générale](#)
- 2. [Structure des fichiers](#)
- 3. [Modes de vue](#)
- 4. [Composants 3D](#)
- 5. [Composants UI \(HTML/CSS\)](#)
- 6. [Stores Zustand](#)
- 7. [Service API](#)
- 8. [Système de caméra](#)
- 9. [Dépendances](#)

1. Architecture générale

L'application est une **Single Page Application** React qui embarque un canvas WebGL Three.js pour le rendu 3D. Elle s'organise autour de trois modes de vue mutuellement exclusifs (système solaire, globe planétaire, ciel nocturne), d'une couche de stores Zustand pour l'état global, et d'un service API centralisé.



2. Structure des fichiers

```

frontend/src/
├─ App.tsx          # Composant racine, canvas, CameraController, routing modes
├─ App.css          # Styles globaux (HUD, panels, overlays, tooltips)
├─ main.tsx         # Point d'entrée React DOM
├─ index.css        # Reset / variables CSS de base
├─
├─ components/      # Composants React
├ │ └─ SolarSystem.tsx # Système solaire complet (Soleil, 8 planètes, orbites, anneaux
Saturne)
├ │   └─ AsteroidBelt.tsx # Ceinture d'astéroïdes (instancedMesh, mode système)
├ │   └─ KuiperBelt.tsx  # Ceinture de Kuiper (instancedMesh, mode système)
├ │   └─ PlanetMoons.tsx # Lunes orbitant une planète (20+ lunes)
├ │   └─ Globe.tsx       # Sphère planète avec shader jour/nuit (CustomShaderMaterial)
├ │   └─ ISS.tsx         # Modèle ISS + orbite (mode globe/Earth, SGP4)
├ │   └─ MarsRovers.tsx  # Marqueurs 3D des 5 rovers martiens
├ │   └─ RoverOverlay.tsx # Overlay plein écran «Mission Control» (lazy-loaded)
├ │   └─ RoverModel3D.tsx # Modèle GLTF rover avec décodeur Draco
├ │   └─ RoverPhotoGallery.tsx # Galerie photos rover (placeholder)
├ │   └─ SatelliteCoverage.tsx # Satellites en orbite (instancedMesh, SGP4, mode globe/Earth)
├ │   └─ PlanetInfoCard.tsx # Panneau info détaillé planète
├ │   └─ Loader3D.tsx     # Écran de chargement global (Suspense)
├ │   └─ NightSky.tsx     # Scène ciel nocturne (mode sky)
├ │   └─ MilkyWay.tsx     # Dôme Voie Lactée (sphère inversée)
├ │   └─ AzAltGrid.tsx    # Grille azimut/altitude
├ │   └─ ConstellationPattern.tsx # Tracé lignes constellation
├ │   └─ ConstellationGuide.tsx # Flèche vers constellation hors-écran
├ │   └─ EarthGuide.tsx   # Indicateur direction du sol
├ │   └─ CompassRose.tsx  # Boussole
├ │   └─ LocationMarker.tsx # Marqueur point d'observation sur globe
├ │   └─ SidePanel.tsx    # Panneau latéral gauche
├ │   └─ ConstellationSidebar.tsx # Barre recherche constellations
├ │   └─ StarTooltip.tsx  # Tooltip étoile survolée
├ │   └─ TimeSlider.tsx   # Curseur temporel
├ │   └─ Effects.tsx      # Post-processing (bloom)
├ │
├ │ └─ stores/          # État global Zustand
├ │   └─ useSkyStore.ts  # Mode de vue, étoiles, planète sélectionnée, rovers, système
solaire
├ │     └─ useConstellationStore.ts # Constellations chargées, sélection
├ │     └─ useObservationStore.ts  # Point d'observation courant
├ │     └─ useISSStore.ts          # Données ISS (TLE, sélection)
├ │     └─ useSatelliteStore.ts    # TLE par groupe, groupes actifs, satellite sélectionné
├ │
├ │ └─ services/
├ │   └─ api.ts                # Client Axios vers le backend

```

```

|
├─ types/                # Interfaces TypeScript
|   ├─ planets.ts        # PlanetId, PLANETS_METADATA (8 planètes)
|   ├─ moons.ts          # MoonData, MOONS_DATA (20+ lunes)
|   ├─ rovers.ts         # RoverMetadata, RoverPosition, RoverFull (5 rovers)
|   ├─ iss.ts            # TLEData, ISSInfo
|   └─ satellite.ts      # SatelliteGroup, SatelliteTLE, SatelliteLiveInfo, SATELLITE_GROUP_META
└─ utils/                # Fonctions utilitaires
    ├─ planetaryEphemeris.ts # Wrapper astronomia → positions 3D planètes
    ├─ dayNightShader.ts    # GLSL shaders terminateur jour/nuit (chunks séparés)
    └─ skyCoords.ts        # Conversions coordonnées célestes

```

3. Modes de vue

L'application possède trois modes exclusifs, contrôlés par `useSkyStore.viewMode` :

Mode "system" — Système Solaire (par défaut)

Caméra : position `[0, 45, 60]` (vue de dessus reculée pour voir l'ensemble du système solaire).

Composants actifs :

- `SolarSystem` — Soleil + 8 planètes avec orbites elliptiques
- `PlanetMoons` — Lunes orbitant autour des planètes gazeuses (Jupiter, Saturne, Uranus, Neptune)
- `PlanetInfoCard` — Panneau détail au clic sur une planète
- Rotation automatique du système solaire (contrôlable via `isSystemRotating`, vitesse en `systemRotationSpeed`)
- Affichage/masquage des orbites (toggle `showOrbits`)

Interactions :

- Clic sur une planète → transition vers le mode "globe" avec zoom sur cette planète
- `TimeSlider` pour modifier le timestamp — les positions et orbites des lunes se mettent à jour

Mode "globe" — Vue Globe (planète détaillée)

Caméra : position `[0, 0, ~2.5]` (distance ajustée selon la taille visuelle de la planète), orbite entre `2.5` et `6` unités.

Planet: Affiche la planète actuellement sélectionnée (par défaut Terre).

Composants actifs :

- `Globe` — sphère texturée (Terre ou autre planète selon sélection) avec shader dynamique jour/nuit
- `ISS` — modèle 3D faible-poly + orbite (pour la Terre uniquement)
- `LocationMarker` — marqueur du point d'observation sur le globe
- `PlanetMoons` — lunes orbitant la planète sélectionnée
- `MarsRovers` — marqueurs 3D des rovers martiens sur Mars
- `PlanetInfoCard` — panneau info détaillé de la planète
- `ConstellationSidebar` — barre de recherche de constellations (HTML, mode Terre uniquement)
- `SidePanel` — panneau latéral (configuration, infos sélection)
- `TimeSlider` — curseur temporel

- `RoverOverlay` — overlay plein écran «Mission Control» (chargé lazy, 3 colonnes : modèle 3D | infos | photos)

Lumière directionnelle : `position=[5, 3, 5]`, `intensity=2` (simule le Soleil depuis la caméra).

Mode "sky" — Vue Ciel Nocturne

Caméra : `position [0, -0.15, 0.1]` (quasi à l'origine, vue 1ère personne). Distance min `0.05`, max infinie. L'utilisateur peut regarder à 360° autour de lui.

Composants actifs :

- `NightSky` — scène principale (étoiles, constellations)
- `MilkyWay` — sphère inversée avec texture équirectangulaire
- `AzAltGrid` — grille azimut/altitude (activable depuis le HUD)
- `ConstellationPattern` — tracé des lignes de constellations
- `ConstellationGuide` — flèche indicatrice vers la constellation sélectionnée
- `EarthGuide` — indicateur de la direction du sol
- `CompassRose` — boussole
- `Effects` — post-processing bloom
- `StarTooltip` — tooltip au survol d'une étoile

HUD top-right : bouton « ← Globe » + toggle de la grille Az/Alt.

4. Composants 3D

Tous les composants listés ici fonctionnent **dans** le `<Canvas>` React Three Fiber et utilisent Three.js directement.

4.1 SolarSystem

`components/SolarSystem.tsx`

Composant principal du mode système solaire. Orchestre le rendu de toutes les planètes, du Soleil, des orbites et des lunes.

Fonctionnalités :

- 8 planètes (Mercure → Neptune) texturées et positionnées via `astronomia`
- Soleil centralisé avec `PointLight`
- Anneaux de Saturne (géométrie plane avec texture map + `DoubleSide` + `alphaMap`)
- Lignes d'orbite elliptiques (optionnelles, contrôlables via `showOrbits`)
- Ceintures d'astéroïdes et de Kuiper (`AsteroidBelt` , `KuiperBelt`)
- Rotation automatique du système (contrôlable via `isSystemRotating`)
- Clic sur une planète → transition vers le mode "globe" avec cette planète

Calcul de positions : via `astronomia` (Meeus Algorithms), temps réel selon le timestamp du store.

Note : `Planet` , `Sun` , `SaturnRings` , `OrbitLine` sont des sous-composants intégrés dans `SolarSystem.tsx` , non exposés comme fichiers séparés.

4.2 AsteroidBelt

`components/AsteroidBelt.tsx`

Représentation 3D de la ceinture d'astéroïdes entre Mars et Jupiter (mode système uniquement).

- `instancedMesh` pour la performance (milliers de points)
 - Distribution aléatoire dans un tore entre ~2.2 et ~3.2 UA
 - Couleur gris-brun semi-transparente
-

4.3 KuiperBelt

`components/KuiperBelt.tsx`

Représentation 3D de la ceinture de Kuiper au-delà de Neptune (mode système uniquement).

- `instancedMesh`, distribution dans un tore entre ~30 et ~55 UA
 - Couleur bleutée semi-transparente
-

4.4 SatelliteCoverage

`components/SatelliteCoverage.tsx`

Satellites en orbite autour de la Terre en temps réel. Monté uniquement si `selectedPlanet === "earth"` ET `showSatellites === true`.

Fonctionnalités :

- Propagation SGP4 via `satellite.js` (`twoline2satrec` + `propagate`) — throttlée à 1×/s
- `instancedMesh` (1 mesh par groupe de couleur) pour la performance (jusqu'à ~5 000 satellites)
- Code couleur par groupe : stations (cyan), Starlink (bleu), GPS (vert), météo (jaune), science (magenta)
- Altitude amplifiée ×2 pour la lisibilité visuelle
- Taille des points en pixels (indépendante du zoom, `sizeAttenuation: false`)
- Satellite sélectionné au clic → `useSatelliteStore.setSelectedSatellite()`
- Chargement des TLE depuis `useSatelliteStore.fetchGroup()` au montage

Groupes affichables : `stations`, `starlink`, `gps-ops`, `weather`, `resource`, `science`, `galileo`

4.5 PlanetMoons

`components/PlanetMoons.tsx`

Système de lunes orbitant une planète donnée (10+ lunes pour Jupiter/Saturne).

Fonctionnalités :

- Orbites dans l'espace local de la planète (rayon = 1)
- Scaling logarithmique pour garder petites lunes lisibles
- Position keplerienne mise à jour avec le timestamp
- Tooltips au survol : nom, distance, période, rayon
- Support des orbites rétrogrades (Triton : `periodDays < 0`)
- Inclinaison orbitale respectée

Lunes disponibles :

- **Terre** : Lune
- **Mars** : Phobos, Deimos
- **Jupiter** : Io, Europa, Ganymède, Callisto
- **Saturne** : Encelade, Téthys, Dioné, Rhéa, Titan

- **Uranus** : Miranda, Ariel, Umbriel, Titania, Obéron
 - **Neptune** : Triton
-

4.6 MarsRovers

`components/MarsRovers.tsx`

Marqueurs 3D des 5 rovers martiens sur la surface de Mars.

Fonctionnalités :

- Marqueurs lumineux (noyau + halo glow)
- Couleur par rover (NASA orange, CNSA rouge, etc.)
- Animation d'échelle au survol/sélection
- Clic → déclenche le RoverOverlay (lazy-loaded)
- Positions dynamiques depuis le backend (`GET /api/rovers/positions`)
- Fallback aux positions par défaut si backend indisponible

Rovers affichés :

- Curiosity (NASA/MSL, actif)
 - Perseverance (NASA/Mars 2020, actif)
 - Opportunity (NASA/MER-B, inactif)
 - Spirit (NASA/MER-A, inactif)
 - Zhurong (CNSA, inactif)
-

4.7 RoverOverlay

`components/RoverOverlay.tsx` (lazy-loaded via `React.lazy()`)

Overlay plein écran déclenché au clic sur un rover. Layout 3 colonnes : modèle 3D | infos | photos.

Structure :

1. **Header** : nom du rover + couleur agence + bouton fermer
2. **Colonne gauche (1/3)** : Canvas R3F avec modèle GLTF rover (rotation auto)
3. **Colonne centre (1.2/3)** : infos mission (site, dates, lat/lon, description)
4. **Colonne droite (1/3)** : galerie photos ou placeholder

Animation :

- Entrée : slide-down depuis le haut (500ms cubic-bezier)
 - Fermeture : reverse + setState delayed
 - Fond : semi-transparent avec blur
-

4.8 RoverModel3D

`components/RoverModel3D.tsx`

Chargement et affichage d'un modèle GLTF rover (support décodeur Draco).

Fonctionnalités :

- Charge fichier GLB depuis `/models/{modelPath}`
- Décodeur Draco à `/draco/`
- Auto-centrage via bounding box

- Auto-scaling pour cible visuelle de 2 unités
 - Fallback cube placeholder si modèle absent
 - Rotation continue ($@\text{delta} \times 0.3$ ou 0.5 rad/s)
-

4.9 Globe

`components/Globe.tsx`

Sphère texturée représentant la Terre. Charge les textures WebP depuis `public/textures/`.

Fonctionnalités :

- Texture couleur + texture normale + texture spéculaire (eau brillante)
 - Rotation automatique selon le timestamp sélectionné (GMST)
 - Clic → déclenche le passage en mode `"sky"` et charge les étoiles pour la position cliquée
-

4.10 ISS

`components/ISS.tsx`

Calcul de position : utilise `satellite.js` (propagateur SGP4) avec les données TLE récupérées depuis `GET /api/iss/tle`.

Fonctionnalités :

- Position calculée côté frontend via SGP4 (mise à jour toutes les secondes)
- Modèle 3D procédural (géométries Three.js primitives assemblées)
- Ligne orbitale : calcul de N points orbitaux sur 90 minutes, rendu `LineLoop`
- Clic → affiche les informations (altitude, vitesse, position) dans le `SidePanel`
- Taille du modèle adaptée au zoom (scale factor)

Store : `useISSStore` (données TLE, sélection, chargement on-demand).

4.11 NightSky

`components/NightSky.tsx`

Scène principale du mode ciel. Orchestre tous les sous-composants.

Fonctionnalités :

- Rendu des étoiles : `Points` Three.js avec taille proportionnelle à la magnitude
 - Couleur des étoiles dérivée de l'indice B-V (rouge → blanc → bleu)
 - Clic/survol sur une étoile → mise à jour de `useSkyStore`
 - Intègre `ConstellationPattern`, `MilkyWay`, `AzAltGrid`, `CompassRose`
 - Recharge les étoiles via `useSkyStore.fetchVisibleStars()` quand la position ou le timestamp change
-

4.12 MilkyWay

`components/MilkyWay.tsx`

Sphère inversée (`side: THREE.BackSide`) avec texture équirectangulaire (2:1) de la Voie Lactée.

- Inclinaison corrigée pour le plan galactique ($\sim 62.9^\circ$)
- Segments suffisants pour éviter la déformation aux pôles

- Opacité réduite pour un rendu atmosphérique

4.13 AzAltGrid

`components/AzAltGrid.tsx`

Grille azimut/altitude en coordonnées horizontales locales.

- Cercles horizontaux tous les 30° d'altitude
- Lignes verticales tous les 30° d'azimut
- Plans opaques sous l'horizon (sol) pour masquer la Voie Lactée au-dessous
- Activable/désactivable via `useSkyStore.showAzAltGrid`

4.14 ConstellationPattern

`components/ConstellationPattern.tsx`

Trace les lignes d'une constellation sélectionnée dans la scène 3D.

- Reçoit les `lines_data` (paires HIP) depuis `useConstellationStore`
- Convertit RA/Dec → position sphérique 3D (rayon unitaire)
- Rendu `<Line>` (drei) avec couleur et opacité configurables

4.15 LocationMarker

`components/LocationMarker.tsx`

Affiche un marqueur sur le globe à la position du point d'observation sélectionné.

5. Composants UI (HTML/CSS)

Ces composants sont rendus **en dehors** du Canvas Three.js, en HTML standard superposé.

5.1 SidePanel

`components/SidePanel.tsx` — ~350 lignes

Panneau latéral gauche. Contenu conditionnel selon l'état :

État	Contenu
Aucune sélection	Formulaire de configuration (ville, magnitude limite, mode switch)
Étoile sélectionnée	Fiche détaillée : nom, magnitude, distance, type spectral, RA/Dec
ISS sélectionnée	Télémétrie ISS : altitude, vitesse, lat/lon
Planète sélectionnée	Infos planète : rayon, masse, période orbitale, description
Rover sélectionné	(Déclenche RoverOverlay à la place)

Actions :

- Sélecteur de point d'observation (liste depuis `/api/observation-points`)

- Slider magnitude limite
 - Bouton « Mode Ciel » → déclenche `setViewMode("sky")` + `fetchVisibleStars()`
-

5.2 ConstellationSidebar

`components/ConstellationSidebar.tsx`

Barre de recherche et liste des constellations (mode globe uniquement).

Fonctionnalités :

- Champ de recherche → appel `GET /api/constellations/search?q=...`
 - Liste des 88 constellations avec nom FR/EN et abréviation
 - Clic → sélectionne la constellation dans `useConstellationStore`
 - Bouton « Meilleur point d'observation » → appel `GET /api/constellations/{id}/best-location`
-

5.3 TimeSlider

`components/TimeSlider.tsx`

Curseur temporel flottant (bas de l'écran, mode sky).

- Entrée `datetime-local` → stocke l'ISO 8601 dans `useSkyStore.timestamp`
 - Bouton « Maintenant » → reset au temps réel
 - Chaque changement déclenche `fetchVisibleStars()` avec le nouveau timestamp
-

5.5 ConstellationGuide

`components/ConstellationGuide.tsx`

Indicateur HTML qui affiche une flèche à l'écran pointant vers la constellation sélectionnée (si hors du champ de vision).

- Utilise `useThree` pour projeter les coordonnées 3D de la constellation en pixels 2D
 - Si dans le champ de vision → affiche un cercle pulsant autour de la position
 - Si hors champ → affiche une flèche sur le bord de l'écran
-

5.6 EarthGuide

`components/EarthGuide.tsx`

Indicateur de la direction du sol en mode sky. S'affiche dans le bas de l'écran quand la caméra pointe vers le haut, et monte quand la caméra pointe vers le bas.

5.7 StarTooltip

`components/StarTooltip.tsx`

Tooltip 3D affiché au survol d'une étoile dans le canvas. Affiche le nom propre (si disponible) et la magnitude.

5.8 Effects

`components/Effects.tsx`

Post-processing `@react-three/postprocessing` :

- Bloom sur les étoiles brillantes (mode sky)
- Bloom sur le Soleil (mode system)

6. Stores Zustand

L'état global est géré avec **Zustand v5** (stores atomiques, sans contexte React).

6.1 useSkyStore

Champ	Type	Description
viewMode	"globe" "sky" "system"	Mode actif (système solaire défaut)
timestamp	string undefined	Temps ISO 8601 sélectionné
stars	VisibleStar[]	Étoiles visibles chargées
loadingStars	boolean	En chargement
hoveredStar	VisibleStar null	Étoile survolée
selectedStar	VisibleStar null	Étoile cliquée
selectedPlanet	PlanetId null	Planète sélectionnée (pour zoom globe)
selectedRoverId	string null	ID du rover (déclenche RoverOverlay)
roverPositions	RoverPosition[]	Positions dynamo des rovers (from backend)
roverOverlayClosing	boolean	Animation fermeture overlay en cours
isSystemRotating	boolean	Mode rotation auto du système solaire
systemRotationSpeed	number	Vitesse en jours/seconde (défaut 15)
showOrbits	boolean	Affichage des orbites planétaires
cameraTarget	[x,y,z] null	Cible caméra (centrage sur étoile)
showAzAltGrid	boolean	Grille Az/Alt visible (mode sky)
error	string null	Message d'erreur

Actions principales :

- `setViewMode(mode) / transitionToMode(mode, planet?)` — bascule de mode avec transition
- `fetchVisibleStars(lat, lon, timestamp?)` — appel API + mise à jour stars
- `fetchRoverPositions()` — récupère positions rovers depuis backend
- `setSelectedRoverId(id)` — sélection rover (déclenche RoverOverlay)
- `setSelectedPlanet(planet)` — sélection planète (zoom sur globe)
- `toggleAzAltGrid()` — toggle grille Az/Alt
- `toggleSystemRotation() / setSystemRotationSpeed(speed)` — contrôle rotation système
- `toggleOrbits()` — affichage/masquage orbites

6.2 useConstellationStore

Champ	Type	Description
constellations	ConstellationListResponse[]	Les 88 constellations
selectedConstellation	ConstellationListResponse null	Constellation active
loading	boolean	—
error	string null	—

Charge les constellations via `GET /api/constellations` au montage.

6.3 useObservationStore

Gère le point d'observation courant (ville sélectionnée dans le SidePanel).

Champ	Type	Description
observationPoints	ObservationPoint[]	Liste complète
selectedPoint	ObservationPoint null	Point actif

6.4 useISSStore

Champ	Type	Description
tleData	TLEData null	Données TLE brutes
issSelected	boolean	ISS est sélectionnée
loading	boolean	Chargement TLE

Stratégie : le TLE n'est récupéré que lors du premier clic sur l'ISS (on-demand), puis mis en cache dans le store.

6.5 useSatelliteStore

Champ	Type	Description
showSatellites	boolean	Toggle global d'affichage (défaut false)
activeGroups	Set<SatelliteGroup>	Groupes actifs (défaut : stations, gps-ops)
tleByGroup	Record<string, SatelliteTLE[]>	TLE cachés par groupe
loadingGroups	Set<string>	Groupes en cours de chargement
error	string null	Erreur de fetch
selectedSatellite	SatelliteLiveInfo null	Satellite sélectionné au clic

Actions :

- `toggleSatellites()` — active/désactive l'affichage global

- `toggleGroup(group)` — ajoute ou retire un groupe des groupes actifs
 - `fetchGroup(group)` — charge les TLE d'un groupe si non déjà en cache
 - `setSelectedSatellite(info)` / `clearSelection()` — sélection satellite
-

7. Service API

`frontend/src/services/api.ts`

Client Axios centralisé. Base URL : `http://localhost:8000` .

Méthodes exposées :

```
// Étoiles & constellation
astronomyService.getVisibleStars(lat, lon, timestamp?, magLimit?)
  → GET /api/stars/visible

astronomyService.getAllConstellations()
  → GET /api/constellations

astronomyService.searchConstellations(q)
  → GET /api/constellations/search?q=...

astronomyService.getBestObservationPoint(constellationId, timestamp?)
  → GET /api/constellations/{id}/best-location

// Points d'observation
astronomyService.getObservationPoints()
  → GET /api/observation-points

// ISS (TLE)
astronomyService.getIssTle()
  → GET /api/iss/tle

// Rovers
astronomyService.getRoverPositions()
  → GET /api/rovers/positions

// Satellites (TLE proxy CelesTrak)
astronomyService.getSatelliteTLEs(group)
  → GET /api/satellites/tle?group={group}

astronomyService.getSatelliteGroups()
  → GET /api/satellites/groups
```

8. Système de caméra

CameraController

Composant interne (dans le Canvas) gérant les transitions de caméra entre modes.

Algorithme de transition :

```
// Lerp exponentiel – fluide et indépendant du framerate
const lerpFactor = 1.0 - Math.exp(-3.0 * delta);

if (viewMode === "sky") {
  camera.position.lerp(SKY_CAMERA_POS, lerpFactor); // [0, -0.15, 0.1]
} else {
  camera.position.lerp(GLOBE_CAMERA_POS, lerpFactor); // [0, 0, 2.5]
}
```

- La transition dure **1.5 secondes** (timer interne), après quoi `useFrame` ne fait plus rien → contrôle total à l'utilisateur.
- Les vecteurs cibles sont **pré-alloués** (`new THREE.Vector3(...)` hors du composant) pour éviter 120 allocations/s dans la boucle de rendu.

OrbitControls

Mode	minDistance	maxDistance
Globe	2.5	6
Sky	0.05	Infinity

`dampingFactor: 0.05` pour une inertie fluide.

9. Dépendances

Paquet	Version	Rôle
react	19.2	UI framework
react-dom	19.2	Rendu DOM
three	0.182	Moteur WebGL 3D
@react-three/fiber	9.5	Bridge React ↔ Three.js
@react-three/drei	10.7	Utilitaires Three.js (OrbitControls, Line, ...)
@react-three/postprocessing	3.0	Effets post-processing (bloom)
zustand	5.0	Gestion d'état global
axios	1.13	Client HTTP
satellite.js	6.0	Propagateur SGP4 (calcul position ISS)
typescript	5.9	Typage statique
vite	7.3	Bundler / dev server
vitest	4.0	Tests unitaires
@testing-library/react	16.3	Tests de composants